

Hodnotící arch pro zkoušejícího

Tento dokument je určen pouze pro zkoušejícího. Nesdílejte ho s uchazečem.

Příprava prostředí

1. Spusťte službu: `npm start` (výchozí port 4010)
2. Ověřte dostupnost: `curl http://localhost:4010/health`
3. Připravte uchazeči notebook s nainstalovaným **Postmanem** a/nebo **SoapUI**
4. Předějte uchazeči soubor `interview-zadani.md`
5. Volitelně importujte Postman kolekci z `postman/interview-mock.postman_collection.json`

Pozor: Služba drží stav v paměti. Po restartu se data vrátí do výchozího stavu. Pokud uchazeč udělá chybu, může být potřeba restartovat službu.

Praktická část: Očekávané odpovědi

Úloha A: REST API

A1 — Healthcheck

```
GET /health → 200
{
  "status": "ok",
  "service": "interview-mock",
  "timestamp": "2026-...",
  "protocols": ["REST", "SOAP"],
  "counts": { "interviews": 2, "candidates": 2 }
}
```

A2 — Seznam pohovorů

```
GET /rest/interviews → 200
Odpověď: { "interviews": [...] } – pole se 2 záznamy (INT-001, INT-002)
Každý záznam musí obsahovat vnořený objekt "candidate" (ne jen candidateId)
```

A3 — Filtrování

```
GET /rest/interviews?status=SCHEDULED → 200
Vrátí pouze INT-001 (status SCHEDULED)
```

A4 — Detail

```
GET /rest/interviews/INT-001 → 200
candidate.firstName = "Anna", candidate.lastName = "Novak"
candidate.skills = ["JavaScript", "REST", "SQL"]
```

A5 — Vytvoření

```
POST /rest/interviews
Content-Type: application/json
{ "candidateId": "CAND-002", "position": "Integration Tester", "scheduledAt": "2026-06-01T10:00:00.000Z" }
→ 201
Nové ID bude INT-003 (auto-increment)
```

A6 — Změna statusu

```
PATCH /rest/interviews/INT-003/status
Content-Type: application/json
{ "status": "IN_PROGRESS" }
→ 200
```

A7 — Hodnocení

```
POST /rest/interviews/INT-003/evaluate
Content-Type: application/json
{ "score": 68 }
→ 200
status = "COMPLETED", recommendation = "REVIEW" (68 je v rozmezí 55–74)
```

A8 — Negativní testy (uchazeč vybírá min. 2)

Scénář	Request	Očekávaná odpověď
Neexistující pohovor	GET /rest/interviews/INT-999	404, INTERVIEW_NOT_FOUND
Neexistující kandidát	POST /rest/interviews { "candidateId": "CAND-999" }	400, UNKNOWN_CANDIDATE
Nevalidní skóre (>100)	POST /rest/interviews/INT-001/evaluate { "score": 150 }	400, INVALID_SCORE
Nevalidní skóre (text)	POST /rest/interviews/INT-001/evaluate { "score": "abc" }	400, INVALID_SCORE
Neplatný status	PATCH /rest/interviews/INT-001/status { "status": "INVALID" }	400, INVALID_STATUS
Neexistující route	GET /rest/neexistuje	404, ROUTE_NOT_FOUND
Neexistující kandidát	GET /rest/candidates/CAND-999	404, CANDIDATE_NOT_FOUND
Interní chyba serveru	(nelze snadno vyvolat)	500, INTERNAL_ERROR

Úloha B: SOAP API

B1 — WSDL

```
GET /soap?wsdl → 200, Content-Type: application/xml
Uchazeč by měl identifikovat 4 operace: ListInterviews, GetInterview, CreateInterview,
UpdateInterviewStatus
```

B2 — ListInterviews

```
POST /soap (Content-Type: text/xml)
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/" xmlns:tns="urn:interview-
mock">
  <soap:Body>
    <tns:ListInterviewsRequest/>
  </soap:Body>
</soap:Envelope>
```

Odpověď: XML s `ListInterviewsResponse`, obsahuje všechny pohovory.

B3 — GetInterview

```
POST /soap (Content-Type: text/xml)
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/" xmlns:tns="urn:interview-
mock">
  <soap:Body>
    <tns:GetInterviewRequest>
      <tns:id>INT-002</tns:id>
    </tns:GetInterviewRequest>
  </soap:Body>
</soap:Envelope>
```

Odpověď: score=82, recommendation=HIRE, candidate.firstName=Petr

B4 — CreateInterview

```
POST /soap (Content-Type: text/xml)
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/" xmlns:tns="urn:interview-
mock">
  <soap:Body>
    <tns:CreateInterviewRequest>
      <tns:candidateId>CAND-001</tns:candidateId>
      <tns:position>SOAP Tester</tns:position>
      <tns:scheduledAt>2026-06-15T14:00:00.000Z</tns:scheduledAt>
    </tns:CreateInterviewRequest>
  </soap:Body>
</soap:Envelope>
```

Odpověď: nové ID (INT-004 pokud již existuje INT-003 z REST úlohy).

B5 — UpdateInterviewStatus

```
POST /soap (Content-Type: text/xml)
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/" xmlns:tns="urn:interview-
mock">
  <soap:Body>
    <tns:UpdateInterviewStatusRequest>
      <tns:id>INT-004</tns:id>
      <tns:status>CANCELLED</tns:status>
    </tns:UpdateInterviewStatusRequest>
  </soap:Body>
</soap:Envelope>
```

Úloha C: Cross-protocol ověření

Uchazeč by měl zavolat `GET /rest/interviews` a najít v odpovědi oba nově vytvořené pohovory (REST i SOAP). To dokazuje, že obě API sdílejí stejný datový stav.

Teoretická část: Očekávané odpovědi

REST otázky

1. REST principy

- Representational State Transfer
- Klíčové principy: client-server, stateless, cacheable, uniform interface, layered system
- Zdroje identifikované URL, manipulace přes HTTP metody

2. PUT vs PATCH

- PUT nahrazuje celý zdroj (úplná reprezentace)
- PATCH aplikuje částečnou modifikaci (pouze změněná pole)
- PUT je idempotentní, PATCH nemusí být (ale v praxi často je)

3. Idempotence

- Opakované volání se stejnými parametry má stejný výsledek jako jedno volání
- Idempotentní: GET, PUT, DELETE, HEAD, OPTIONS
- Ne-idempotentní: POST (vytváří nový zdroj při každém volání)

4. HTTP status kódy

- 1xx: Informační (100 Continue, 101 Switching Protocols)
- 2xx: Úspěch (200 OK, 201 Created, 204 No Content)
- 3xx: Přesměrování (301 Moved Permanently, 302 Found, 304 Not Modified)
- 4xx: Chyba klienta (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found)
- 5xx: Chyba serveru (500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable)

5. OpenAPI/Swagger

- Standardizovaný formát popisu REST API (YAML/JSON)
- Definuje endpointy, metody, parametry, request/response schémata
- Umožňuje generování dokumentace, klientů, serverových stubů a testů

6. Autentizace/autorizace

- API klíče: jednoduché, v headeru nebo query parametru
- OAuth 2.0: autorizační framework s tokeny (access/refresh)
- JWT: self-contained tokeny s claims, podepsané
- Testování: platný/neplatný/expirovaný token, nedostatečná oprávnění

7. Content Negotiation

- Mechanismus HTTP pro dohodnutí formátu odpovědi
- Klient posílá `Accept` header (např. `application/json`, `application/xml`)
- Server posílá `Content-Type` header s formátem odpovědi

SOAP otázky

8. SOAP struktura

- Simple Object Access Protocol
- Envelope: kořenový element, definuje namespaces
- Header: volitelný, metadata (autentizace, routing)
- Body: povinný, obsahuje samotnou zprávu/operaci
- Fault: volitelný (v Body), chybové informace

9. WSDL

- Web Services Description Language
- Obsahuje: typy (XSD schémata), zprávy, operace, port types, bindings, services
- Definuje „kontrakt“ služby — jaké operace jsou dostupné a jak je volat

10. Document/literal vs RPC/encoded

- Document/literal: XML dokument v Body, validovatelný přes XSD, moderní standard
- RPC/encoded: vzdálené volání procedury, parametry jako elementy s typy
- Document/literal je dnes preferovaný (WS-I Basic Profile)

11. SOAP Fault

- Element v SOAP Body pro chybové odpovědi

- Struktura: faultcode (kód chyby), faultstring (popis), detail (dodatečné info)
- SOAP 1.2: Code, Reason, Detail, Node, Role

12. SOAP vs REST

- SOAP výhody: formální kontrakt (WSDL), WS-Security, transakce (WS-AtomicTransaction), spolehlivost (WS-ReliableMessaging)
- SOAP nevýhody: verbose XML, složitější implementace, horší výkon
- REST výhody: jednoduchost, flexibilní formáty (JSON), cachování, široká podpora
- REST nevýhody: žádný formální kontrakt (OpenAPI je volitelný), méně standardizovaná bezpečnost

Obecné otázky

13. Integrační testování

- Testuje interakci mezi komponentami/systémy
- Unit: izolovaná jednotka kódu
- Integrační: spolupráce více komponent, reálné závislosti
- E2E: celý systém end-to-end včetně UI

14. Testování s třetí stranou

- Mockování/stubování externí služby v nižších prostředích
- Contract testing pro ověření kompatibility
- Sandbox/testovací prostředí poskytovatele
- Monitoring a alerting na produkci
- Testování timeoutů, retries, fallbacků

15. Contract testing

- Ověřuje kompatibilitu API mezi consumer a provider
- Pact: consumer-driven contracts
- Vhodné pro microservices, kdy více týmů vyvíjí nezávisle
- Consumer definuje očekávání, provider je verifikuje

16. Výkonnostní testování

- Zátěžové testy: chování pod zátěží (JMeter, k6, Gatling)
- Stress testy: hledání bodu zlomu
- Soak testy: dlouhodobá stabilita
- Metriky: response time, throughput, error rate, percentily (p50, p95, p99)

17. Pozitivní vs negativní testování

- Pozitivní: validní vstupy → očekávaný úspěšný výsledek
- Negativní: nevalidní vstupy → správná chybová odpověď
- API příklady: pozitivní = validní JSON → 201; negativní = chybějící pole → 400

18. Mocky a stuby

- Stub: vrací předem definované odpovědi (pasivní)
- Mock: ověřuje, že byl zavolán se správnými parametry (aktivní, assertions)
- Vhodné: izolace od externích závislostí, nestabilní služby, vývoj paralelně

Doporučený průběh pohovoru

1. **Úvod (5 min)** — představení, vysvětlení formátu
2. **Praktická část (60 min)** — uchazeč pracuje, zkoušející pozoruje a odpovídá na dotazy k zadání
3. **Teoretická část (15 min)** — zkoušející vybere 2 REST + 2 SOAP + 2 obecné otázky
4. **Diskuze (10 min)** — uchazeč prezentuje shrnutí, zkoušející se ptá na postřehy
5. **Závěr (5 min)** — prostor pro otázky uchazeče

Na co se zaměřit při pozorování:

- Systematičnost přístupu (nejdřív si přečte dokumentaci, nebo rovnou zkouší?)
- Kvalita assertions (ověřuje jen status kód, nebo i tělo odpovědi?)
- Organizace práce (pojmenovávání requestů, struktura kolekce)
- Schopnost interpretovat chybové odpovědi
- Práce s WSDL a XML (zvládá namespace, strukturu envelope)
- Komunikace — jak popisuje nalezené problémy